



The Lazy Developer

22

MODULE

AI Agents: From Concept to Production (Advanced Tutorial)

A deep dive into building and deploying AI agents. Understand agentic workflows, design agent architectures, and learn to build agents with Claude, OpenAI, and open-source models.

Table of Contents

- 01. Understanding AI agents and agentic workflows (22 min)
- 02. Designing agent architectures (24 min)
- 03. Building agents with Claude, OpenAI, and open-source models (25 min)
- 04. Testing and deploying AI agents (23 min)

Understanding AI Agents and Agentic Workflows

In Module 20, we introduced AI agents as the personal chef who grocery shops, cooks, and plates the meal while you simply say "something Italian tonight." You learned the basic agentic loop - perceive, plan, act, observe, repeat -- and compared the major platforms.

Now it is time to open the hood and understand how that chef actually thinks. Because once you move from using agents to building them, the difference between a clumsy bot and a genuinely useful assistant comes down to the workflow patterns, memory systems, and reasoning strategies running under the surface.

Think of it this way: Module 20 taught you to drive the car. This module teaches you how the engine works so you can build your own.

From Simple Chains to Agentic Workflows

Before we go deeper, let's clarify an important distinction that trips people up.

A chain is a fixed sequence of steps. Input goes in, passes through Step A, then Step B, then Step C, and output comes out. Every run follows the exact same path. Most "AI automation" you see on YouTube is actually a chain - it looks impressive, but it cannot adapt when something unexpected happens.

An agentic workflow is a dynamic sequence. The agent decides which step comes next based on what it observes. It can loop back, skip ahead, call for help, or abandon a path that is not working. The route is not pre-programmed; it emerges from the agent's reasoning.

Here is a concrete example:

Scenario	Chain Approach	Agentic Workflow Approach
"Summarise this PDF"	Extract text -> Send to LLM -> Return summary	Extract text -> Check if text is readable -> If scanned image, run OCR first -> Send to LLM -> Form hypothesis -> Apply fix -> Evaluate summary quality -> If poor, retry with different prompt -> Return summary
"Fix this bug"	Paste error into LLM -> Return suggested fix	Read error -> Read related files -> Send to LLM -> Form hypothesis -> Apply fix -> Evaluate summary quality -> If poor, retry with different prompt -> Return summary
"Research competitors"	Send query to search API -> Summarise results	Send query to search API -> Summarise results -> Identify gaps -> Search again -> Compile structured report

The chain is a conveyor belt. The agentic workflow is a thinking process. That distinction matters enormously when you start building your own systems.

The Agentic Loop in Detail: Observe, Think, Act, Reflect

In Module 20 we introduced the loop as "Perceive, Plan, Act, Observe, Repeat." Let's refine that into the four-phase model that most agent frameworks actually implement:

```
+---> OBSERVE ----> THINK ----> ACT ----> REFLECT ----+
|
+----- LOOP BACK <-----+
```

1. Observe -- The agent gathers information. This could mean reading files, querying a database, calling an API, parsing a web page, or simply re-reading the user's original request. Observation is the agent's sensory input.
2. Think -- The agent reasons about what it observed. This is where the LLM shines as it evaluates the situation, considers options, and decides what to do next. In framework terms, this is often called the "planning" or "reasoning" step.
3. Act -- The agent executes a concrete action. It might write a file, call a tool, send a message, or generate a response. Actions change the state of the world.
4. Reflect -- The agent evaluates the result of its action. Did it work? Did it move closer to the goal? Should it try something different? Reflection is what separates a genuine agent from a script that blindly runs steps in order.

The key insight is the loop. After reflecting, the agent goes back to observe the new state of the world and decides what to do next. This continues until the agent determines the goal has been met (or it gives up after too many attempts).

Why this matters for you: When you build an agent, you are designing each phase of this loop. What can the agent observe? What tools can it use to act? How does it decide when to stop? These are the engineering decisions that determine whether your agent is useful or frustrating.

The ReAct Pattern: How Modern Agents Reason

If you read any AI agent documentation, you will encounter the term ReAct within the first few pages. It stands for Reasoning + Acting, and it is the dominant pattern for how LLM-based agents work today.

The idea is elegantly simple: instead of having the agent just take actions, you force it to write out its reasoning before each action. This makes the agent more reliable because the reasoning step helps it think through the problem before jumping to a solution.

Here is what a ReAct loop looks like in practice:

User: Find the total revenue from last quarter and email it to Sarah.

Agent Thought: I need to find last quarter's revenue. I should check the database for sales records from Q3. Let me query the sales table.

Agent Action: `query_database("SELECT SUM(amount) FROM sales WHERE quarter = 'Q3_2025'")`

Agent Observation: Result: \$247,500

Agent Thought: I have the revenue figure: \$247,500. Now I need to email this to Sarah. Let me compose a professional email with this information.

Agent Action: `send_email(to="sarah@company.com", subject="Q3 Revenue Summary", body="Hi Sarah, the total revenue for Q3 2025 was $247,500.")`

Agent Observation: Email sent successfully.

Agent Thought: Both tasks are complete. The revenue was retrieved and the email was sent to Sarah. I can provide a final summary.

Agent Response: Done! Q3 revenue was \$247,500 and I've emailed the summary to Sarah.

Notice the pattern: Thought -> Action -> Observation, repeated until the task is complete. The "Thought" step is the magic ingredient -- it gives the LLM a scratchpad to reason before committing to an action.

Why ReAct works so well:

- Forces the agent to plan before acting (fewer mistakes)
- Creates a visible chain of reasoning (easier to debug)
- Allows the agent to course-correct mid-task
- Produces a natural audit trail of what happened and why

Every major agent framework -- LangChain, CrewAI, Claude's agent capabilities, OpenAI's Agents SDK -- implements some variation of ReAct under the hood.

Tool Use: Giving Agents Hands

An LLM without tools is a brain in a jar. It can think, but it cannot do anything. Tools are what turn a language model into an agent.

When we say "tools," we mean any function the agent can call to interact with the outside world:

Tool Category	Examples	What It Enables
File System	Read file, write file, list directory	Agent can work with your codebase
Terminal/Shell	Run commands, install packages, execute scripts	Agent can build and test software
Database	Query, insert, update, delete records	Agent can work with structured data
Web/API	HTTP requests, web search, scrape pages	Agent can gather external information
Communication	Send email, post to Slack, create GitHub issue	Agent can interact with people and services
Code Execution	Run Python/JavaScript, evaluate expressions	Agent can compute and validate

When you build an agent, you define which tools it has access to. This is one of the most important design decisions you will make. Too few tools and the agent is helpless. Too many tools and the agent gets confused about which one to use.

Claude prompt you could use right now:
I want to build an AI agent that can help me manage my project tasks. What tools would this agent need access to?
Think about reading/writing files, accessing a database, and sending notifications. List each tool with its purpose.

Agent Memory: Short-Term vs Long-Term

Human brains have working memory (what you are thinking about right now) and long-term memory (things you learned years ago). Agents need both too, and understanding memory is critical for building agents that do not forget what they were doing halfway through a task.

Short-Term Memory (Context Window)

This is the conversation history where everything the agent has seen and done during the current session. It is stored in the LLM's context window, which has a fixed size limit.

The problem: Context windows are large but not infinite. Claude's context window is (as of this writing) 200K tokens (roughly 150,000 words). GPT-5 is about the same. That sounds like a lot, but when your agent is reading files, receiving tool outputs, and accumulating reasoning steps, it fills up fast.

What happens when memory fills up: The agent starts "forgetting" earlier parts of the conversation. It might re-read a file it already read, contradict an earlier decision, or lose track of the original goal. This is one of the most common failure modes in agent systems.

Strategies for managing short-term memory:

- Summarise earlier steps periodically (condense 5 pages of reasoning into a paragraph)
- Only keep the most relevant tool outputs (discard raw data after extracting key facts)
- Break large tasks into subtasks that each fit comfortably in context

Long-Term Memory (Persistent Storage)

This is information that persists between sessions. It is stored outside the LLM in a database, vector store, or file system.

Examples of long-term memory:

- User preferences ("Sarah prefers bullet-point summaries")
- Project context ("This codebase uses React with TypeScript and Tailwind")
- Past interactions ("Last week we decided to use Supabase instead of Firebase")
- Knowledge base (documentation, FAQs, company policies)

How it works in practice: Before the agent starts reasoning, a retrieval step searches long-term memory for relevant context and injects it into the prompt. This is often called RAG (Retrieval-Augmented Generation) -- the agent retrieves relevant memories before generating its response.

Memory Type	Where It Lives	Lifespan	Size Limit	Example
Short-term	LLM context window	Current session only	128K-200K tokens	"I just read the auth.ts file and found a bug on line 42"
Long-term	Database / Vector store	Persists across sessions	Practically unlimited	"This user's project uses Express + Supabase on Railway"
Episodic	Structured logs	Persists across sessions	Practically unlimited	"Last Tuesday the agent failed to deploy because of a missing env variable"

Multi-Agent Systems: When One Agent Is Not Enough

Sometimes a single agent trying to do everything is like one person trying to be the architect, plumber, electrician, and interior designer on a house renovation. It is possible, but the results will not be great.

Multi-agent systems use multiple specialised agents that collaborate to complete a task. Each agent has its own role, tools, and expertise.

Here is a simple example -- imagine you ask a system to "build a landing page for my new product":

```
Orchestrator Agent: Receives the request, breaks it down into subtasks.
|
+-- Designer Agent: Creates the layout and visual structure.
|   Tools: design templates, colour palette generator
|
+-- Copywriter Agent: Writes the headlines, body text, and CTAs.
|   Tools: brand guidelines, product documentation
|
+-- Developer Agent: Turns the design and copy into actual code.
|   Tools: file system, terminal, code execution
|
+-- QA Agent: Reviews the final page for issues.
    Tools: browser, accessibility checker, link validator
```

Each agent focuses on what it does best. The orchestrator coordinates the workflow, passes outputs from one agent to the next, and handles any conflicts or issues.

Why multi-agent systems matter for you:

- They can tackle problems too complex for a single agent
- Each agent can use a different model (cheap model for simple tasks, powerful model for complex reasoning)
- They mirror how real teams work, making them intuitive to design
- They provide natural checkpoints where humans can review intermediate results

We will go much deeper into multi-agent architecture patterns in the next section. For now, just know that when you hear "multi-agent," think "team of specialists" rather than "one genius doing everything."

Agent Planning and Reasoning

The difference between a good agent and a bad agent often comes down to how well it plans.

A naive agent charges at the first subtask it identifies. A sophisticated agent steps back, considers the full scope of the problem, and creates a plan before taking any action.

Here are the main planning strategies agents use:

1. Upfront Planning -- The agent creates a complete plan before taking any action. Good for well-defined tasks where the steps are predictable.

```
User: Migrate the database from SQLite to PostgreSQL.
```

```
Agent Plan:
```

- ```
1. Audit current SQLite schema (tables, columns, constraints)
2. Generate equivalent PostgreSQL schema
3. Write data migration script
4. Update application connection strings
5. Run migration on test database
6. Verify data integrity
7. Update environment variables for production
```

2. Iterative Planning -- The agent plans one or two steps ahead, executes them, and then re-plans based on what it learned. Good for tasks with uncertainty or when the full scope is not clear at the start.

3. Hierarchical Planning -- The agent creates high-level goals, then decomposes each goal into subtasks, then decomposes those further. Good for complex, multi-day projects.

```
High-level: Build an e-commerce checkout system
-> Sub-goal: Shopping cart functionality
 -> Task: Create cart data model
 -> Task: Build add-to-cart API endpoint
 -> Task: Build cart UI component
-> Sub-goal: Payment processing
 -> Task: Integrate Stripe SDK
 -> Task: Build checkout form
 -> Task: Handle webhook events
```

The planning strategy you choose (or that your agent framework chooses) has a huge impact on reliability. We will cover how to implement these patterns in Section 2.

---

## How This All Connects: An End-to-End Example

Let's put everything together with a realistic scenario. Imagine you have built a project management app, and you want an AI agent to handle customer support tickets:

1. Ticket arrives: "I can't export my project data to CSV."
2. Observe: Agent reads the ticket, checks user's account details, retrieves the export feature documentation from long-term memory.
3. Think (ReAct): "This could be a bug or a permissions issue. Let me check the user's subscription tier first, since CSV export might be a premium feature."
4. Act (Tool Use): Agent queries the database for the user's subscription tier.
5. Observe: User is on the free tier. CSV export requires the Pro tier.
6. Reflect: "This is not a bug, it is a feature limitation. I should explain this politely and offer an upgrade path."
7. Act: Agent composes a response explaining the limitation and includes a link to upgrade.
8. Reflect: "Response sent. The ticket can be marked as resolved."

Notice how every concept we covered from the agentic loop, ReAct reasoning, tool use, and memory shows up naturally in this single interaction. That is what makes these concepts so important: they are not academic abstractions. They are the building blocks of every agent you will build.

---

## AI Prompt Examples

Try these prompts in Cursor or Claude Code to explore agentic concepts:



Explain the difference between a simple LLM chain and an agentic workflow. Give me a concrete example of a task that requires an agentic approach and would fail with a simple chain.

Design a ReAct-style reasoning loop for an agent that monitors my website's uptime. Show me the Thought/Action/Observation steps it would take when it detects the site is down.

I want to build an agent with long-term memory. What are my options for storing and retrieving memories? Compare vector databases, traditional databases, and file-based approaches.

Sketch out a multi-agent system for automating my weekly project status reports. What agents would I need, what tools would each agent have, and how would they coordinate?

---

## Key Takeaways

- Agentic workflows are dynamic, not fixed -- the agent decides what to do next based on what it observes, unlike chains that follow a pre-set sequence every time.
- The agentic loop (Observe, Think, Act, Reflect) is the core engine of every AI agent. Understanding this loop is essential for building agents that actually work.
- ReAct (Reasoning + Acting) forces the agent to think before it acts, making it more reliable and easier to debug. Every major framework uses some version of this pattern.
- Tools are what separate an agent from a chatbot. Without tools, an LLM can only generate text. With tools, it can read files, query databases, send emails, and interact with the real world.
- Memory management is critical. Short-term memory (context window) fills up fast; long-term memory (databases, vector stores) lets agents remember across sessions. Design both carefully.
- Multi-agent systems let you break complex problems into specialised roles, just like a real team. One orchestrator coordinates multiple specialist agents.
- Planning strategy matters. Upfront planning works for predictable tasks; iterative planning works for uncertain ones; hierarchical planning works for complex projects. Choose wisely.
- All of these concepts work together in every real agent interaction. The loop, the reasoning, the tools, the memory, they are not separate features but interconnected parts of one system.

## Designing Agent Architectures

Imagine you are planning a wedding. You could try to do everything yourself: book the venue, arrange the flowers, hire the DJ, design the invitations, coordinate the catering, manage the guest list. Some people do this, and some of those weddings turn out beautifully. But most people who try the solo approach end up stressed, overwhelmed, and serving slightly cold chicken.

The alternative? Hire a wedding planner who coordinates a team of specialists. The florist handles flowers. The caterer handles food. The DJ handles music. The planner makes sure everyone shows up at the right time and nothing falls through the cracks.

Designing an agent architecture is exactly the same decision. You need to figure out: should one agent do everything, or should you build a team? And if you build a team, how should they communicate, who is in charge, and what happens when something goes wrong?

This section gives you the blueprints.

---

### Why Architecture Matters

Before we look at specific patterns, let's understand why you cannot just throw an LLM at a problem and hope for the best.

Without good architecture, agents:

- Lose track of what they are doing on complex tasks
- Use the wrong tool at the wrong time
- Get stuck in infinite loops (trying the same failed approach repeatedly)
- Cost you money by making unnecessary API calls
- Produce inconsistent results that you cannot trust

With good architecture, agents:

- Break complex problems into manageable pieces
- Recover gracefully from errors
- Stay within budget by using the right model for each subtask
- Produce reliable, repeatable results
- Can be debugged and improved over time

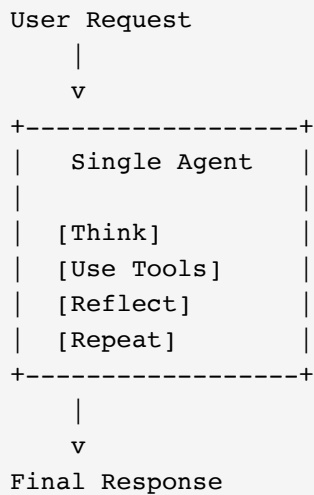
Think of architecture as the difference between a pile of LEGO bricks and an actual LEGO house. Same pieces, vastly different outcomes.

---

## Pattern 1: Single Agent (The Solo Performer)

This is the simplest architecture and the one you should start with.

How it works: One agent receives a task, reasons about it, uses tools to complete it, and returns the result. There is no delegation, no coordination, no handoffs. Just one LLM in a loop.



When to use it:

- The task is well-defined and can be completed in under ~20 steps
- You do not need different expertise for different parts of the task
- You want simplicity and fast iteration
- You are building your first agent (always start here)

When it breaks down:

- Tasks that require switching between very different skill sets (writing code AND writing marketing copy)
- Tasks that are so long the agent runs out of context window
- Tasks that need parallel execution (doing multiple things at the same time)

Real-world example: A customer support agent that reads a ticket, checks the knowledge base, and writes a response. One agent, one skill set, one workflow.

*AI prompt to try:*

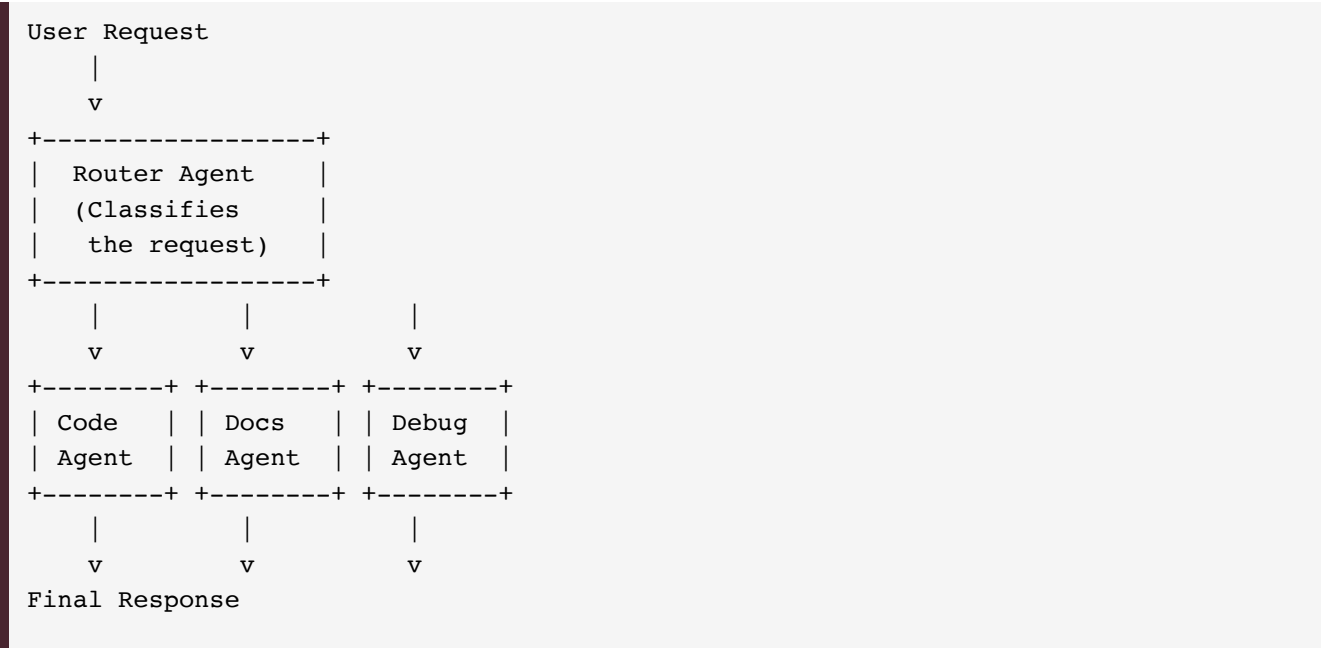
*Build me a simple single-agent system in Python that can answer questions about my codebase. It should be able to read files, search for text, and list directory contents.*

*Use a ReAct loop with Thought/Action/Observation steps.*

## Pattern 2: Router (The Traffic Director)

Sometimes you need different agents for different types of tasks, but only one agent works at a time. The router pattern solves this.

How it works: A lightweight "router" agent receives every request and decides which specialised agent should handle it. The request gets forwarded to the appropriate agent, which does the actual work.



When to use it:

- You handle several distinct types of requests
- Each type benefits from a different system prompt, tool set, or model
- You want to keep each agent simple and focused
- You need to route between "fast and cheap" vs "slow and powerful"

Design decisions:

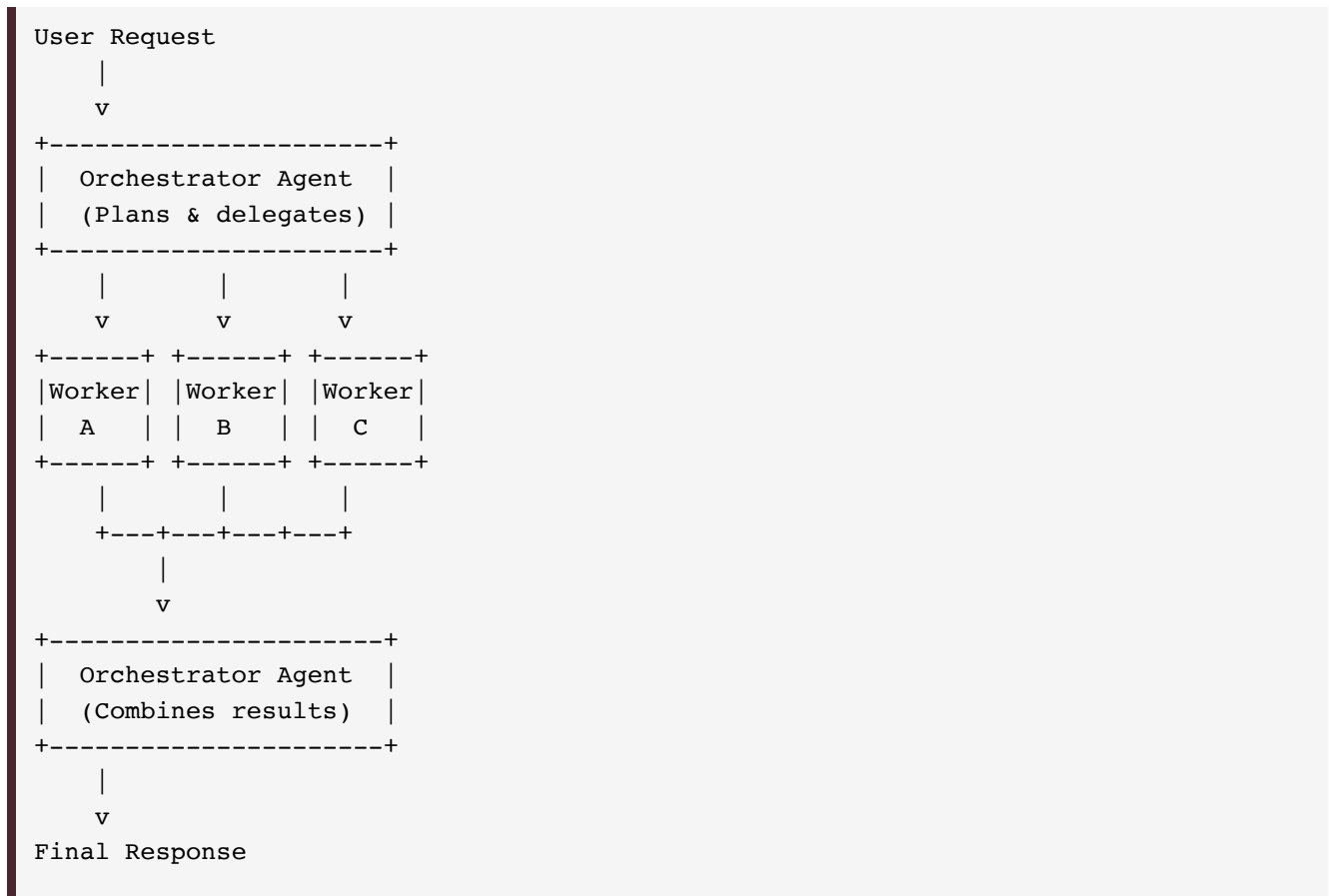
| Decision                      | Options                                                             | Trade-off                                                                     |
|-------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------|
| How does the router classify? | LLM-based classification, keyword matching, or embedding similarity | LLM is most flexible but adds latency and cost; keywords are fast but brittle |
| What if the router is wrong?  | Add a fallback agent, allow re-routing, or ask the user to clarify  | Fallbacks add reliability but increase complexity                             |
| Different models per agent?   | Claude Haiku for simple tasks, Claude Opus for complex reasoning    | Saves money but requires careful classification                               |

Real-world example: An internal company assistant. Questions about HR policies go to the HR agent. Questions about IT issues go to the IT agent. Code review requests go to the code agent. The router figures out who should answer.

### Pattern 3: Orchestrator-Worker (The Project Manager)

This is the most common multi-agent pattern, and it is the one you will use most often for anything beyond simple tasks.

How it works: An orchestrator agent receives the task, breaks it into subtasks, delegates each subtask to a specialised worker agent, collects the results, and assembles the final output.



When to use it:

- The task naturally decomposes into independent subtasks
- Different subtasks need different tools or expertise
- You want human review at the orchestration level
- The task is complex enough that a single agent would lose context

The orchestrator's job is critical. A good orchestrator:

1. Analyses the task and creates a plan
2. Decides which workers to invoke and in what order
3. Passes the right context to each worker (not too much, not too little)
4. Handles failures (retries, alternative workers, escalation)
5. Synthesises worker outputs into a coherent final result

Design decisions:

| Decision                               | Options                                                                | Trade-off                                                                                                                               |
|----------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Sequential vs parallel workers?        | Run workers one at a time or simultaneously                            | Parallel is faster but harder to coordinate; sequential allows each worker to build on previous results                                 |
| How much context does each worker get? | Full task + all previous outputs, or just their specific subtask       | More context = better results but fills up the context window; less context = faster and cheaper but workers may miss important details |
| What if a worker fails?                | Retry, use a different worker, ask the orchestrator to adjust the plan | Retries are simple; re-planning is more robust but slower                                                                               |
| Which model for the orchestrator?      | Always use the most capable model                                      | The orchestrator needs strong reasoning; workers can often use cheaper models                                                           |

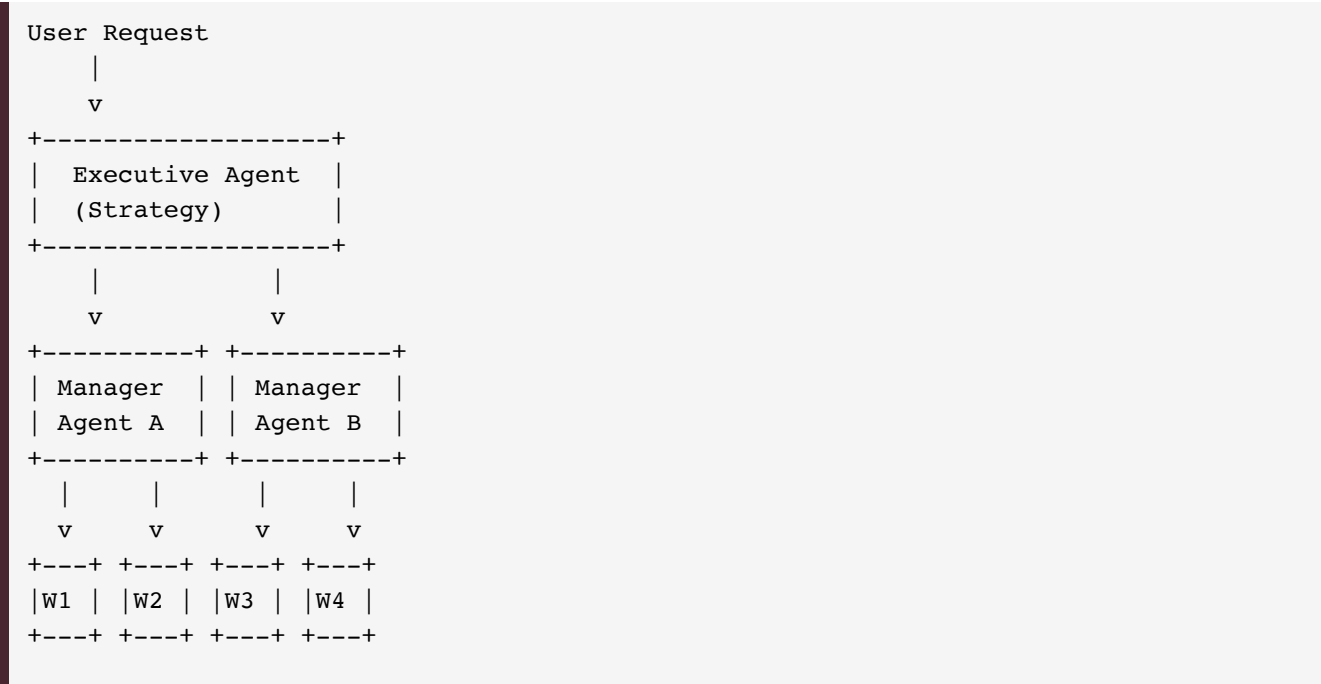
Real-world example: Building a blog post. The orchestrator breaks the task into: research the topic (Research Worker), write the draft (Writing Worker), create illustrations (Image Worker), format and optimise for SEO (SEO Worker). The orchestrator reviews each output and asks for revisions if needed before assembling the final post.

*AI prompt to try:*

*Design an orchestrator-worker agent system for processing customer feedback. The orchestrator should classify feedback into categories, then delegate to specialist workers: one for bug reports, one for feature requests, and one for general complaints. Show me the system prompt for the orchestrator and one worker.*

Pattern 4: Hierarchical Agents (The Corporate Org Chart)

Think of this as the orchestrator-worker pattern with multiple levels. You have a top-level agent that delegates to mid-level agents, which in turn delegate to lower-level agents.



When to use it:

- Very large, complex projects with multiple phases
- Tasks that naturally have a hierarchy (strategy -> tactics -> execution)
- When you need different levels of oversight at different stages
- Enterprise applications where accountability and audit trails matter

When it is overkill:

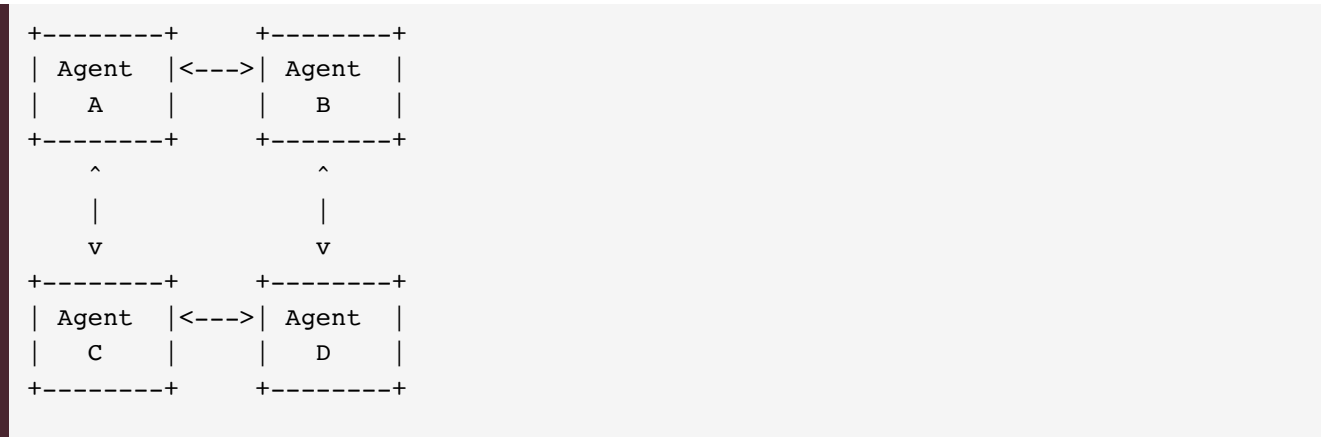
- Most tasks. Seriously, do not reach for this pattern unless you genuinely need it. Two levels (orchestrator + workers) handle 90% of real-world use cases.

Real-world example: An automated software development pipeline. The Executive Agent decides what features to build based on user feedback. Manager Agent A handles frontend features, delegating to workers that write React components, CSS, and tests. Manager Agent B handles backend features, delegating to workers that write API routes, database migrations, and documentation.

### Pattern 5: Swarm (The Bee Colony)

This is the newest and most experimental pattern. Instead of a rigid hierarchy, agents form a flexible network where control passes dynamically based on the situation.

How it works: Agents are loosely connected. When one agent determines it is not the best fit for the current subtask, it hands off to a more suitable agent. There is no central coordinator -- the agents self-organise.



Key concept: Handoffs. In a swarm, agents can "hand off" the conversation to another agent when they recognise the task has shifted into another agent's area of expertise.

For example, a user starts asking about a billing issue (Billing Agent handles it). Mid-conversation, the user mentions a bug in the checkout flow (Billing Agent hands off to Engineering Agent). The Engineering Agent investigates and discovers the bug is in the payment integration (hands off to Payments Agent).

When to use it:

- Customer-facing applications where conversations naturally shift topics
- Systems where you cannot predict the workflow in advance
- When you want agents to collaborate without a central bottleneck

When to avoid it:

- When you need predictable, auditable workflows
- When tasks have a clear sequential structure
- When you are just getting started (swarms are hard to debug)

Real-world example: OpenAI's Swarm framework and Anthropic's multi-agent patterns both support this approach. Think of a virtual assistant that can handle travel bookings, calendar management, email drafting, and expense reporting are each handled by a specialist, with smooth handoffs between them.

## Choosing the Right Architecture

Here is the decision framework you should actually use:

| Question                                                             | If Yes              | If No                                                      |
|----------------------------------------------------------------------|---------------------|------------------------------------------------------------|
| Can one agent handle the entire task in under 20 steps?              | Single Agent        | Continue...                                                |
| Are there distinct categories of tasks that need different handling? | Router              | Continue...                                                |
| Can the task be broken into independent subtasks?                    | Orchestrator-Worker | Continue...                                                |
| Do subtasks have their own subtasks (multiple levels)?               | Hierarchical        | Continue...                                                |
| Does the workflow shift dynamically and unpredictably?               | Swarm               | Go back and reconsider -- you might be overcomplicating it |

The golden rule: Start simple, add complexity only when the simple approach fails. A well-designed single agent beats a poorly designed multi-agent system every time.

## State Management: Keeping Track of Everything

No matter which architecture you choose, you need to manage state which is the running record of what has happened, what is happening, and what still needs to happen.

What state includes:

- The original task and any clarifications
- The current plan (and any revisions to it)
- Which steps have been completed and their results
- Which steps are in progress



- Any errors encountered and how they were handled
- The accumulated context that agents need to do their work

State management approaches:

| Approach          | How It Works                                                               | Best For                                                                     |
|-------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------------|
| In-context        | State is just the conversation history                                     | Single agent, short tasks                                                    |
| Structured object | A JSON/dictionary that gets passed between agents and updated at each step | Orchestrator-worker, medium complexity                                       |
| Database          | State is persisted to a database (PostgreSQL, Redis)                       | Production systems, long-running tasks, systems that need to survive crashes |
| Event log         | Every action and observation is recorded as an immutable event             | Audit-heavy environments, debugging complex multi-agent systems              |

For most projects you will build, a structured state object is the sweet spot.

```
Example state object for an orchestrator-worker system
state = {
 "task": "Build a landing page for ProductX",
 "plan": [
 {"step": 1, "description": "Research competitor pages", "status": "completed"},
 {"step": 2, "description": "Write copy", "status": "in_progress", "assigned_to": "C9"},
 {"step": 3, "description": "Build HTML/CSS", "status": "pending"},
 {"step": 4, "description": "Review and QA", "status": "pending"}
],
 "errors": [],
 "context": {
 "brand_guidelines": "...",
 "target_audience": "non-technical founders"
 }
}
```

## Error Handling and Recovery

Agents fail. Models hallucinate. APIs time out. Tools return unexpected results. Your architecture needs to handle all of this gracefully.

Common failure modes and how to handle them:

| Failure                 | What Happens                                       | Recovery Strategy                                                                                        |
|-------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Tool call fails         | API returns an error or times out                  | Retry with exponential backoff (wait 1s, 2s, 4s...). After 3 retries, stop the step and note the failure |
| Agent goes in circles   | Keeps trying the same approach that is not working | Set a maximum iteration count. If hit, force a re-plan or escalate to human                              |
| Context window overflow | Too much accumulated context                       | Summarise earlier steps, drop irrelevant tool outputs, or split into a new session                       |
| Hallucination           | Agent invents facts or tool outputs                | Validate tool outputs against expected schemas. Use a separate checker agent for cannot handle           |
| Wrong agent selected    | Router sends task to the wrong specialist          | Allow agents to reject tasks they cannot handle and route back                                           |
| Cascading failure       | One worker's failure breaks the entire pipeline    | Isolate workers so one failure does not block others. Use circuit breakers                               |

The most important principle: Always have a fallback. If the agent cannot complete the task autonomously, it should gracefully hand off to a human rather than producing garbage output.

```
Pseudocode for basic error handling in an agent loop
max_iterations = 15
iteration = 0

while not task_complete and iteration < max_iterations:
 try:
 observation = agent.observe()
 thought = agent.think(observation)
 result = agent.act(thought)
 task_complete = agent.reflect(result)
 except ToolError as e:
 agent.log_error(e)
 agent.retry_or_skip()
 except ContextOverflow:
 agent.summarize_and_compress()

 iteration += 1

if not task_complete:
 agent.escalate_to_human("Could not complete task after max iterations")
```

## Guardrails and Safety

Agents that can read files, run commands, and call APIs are powerful. They are also potentially dangerous. Guardrails are the safety mechanisms that prevent agents from doing things they should not do.

Essential guardrails:

| Guardrail         | What It Prevents                                         | How to Implement                                                                                   |
|-------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Tool restrictions | Agent deleting files it should not touch                 | Whitelist allowed tools and allowed file paths                                                     |
| Output validation | Agent generating harmful or inappropriate content        | Run outputs through a content filter before returning to user                                      |
| Rate limiting     | Agent making thousands of API calls and running up costs | Set per-minute and per-session limits on tool calls                                                |
| Human-in-the-loop | Agent making irreversible changes without approval       | Require confirmation for destructive actions (delete, deploy, send email)                          |
| Budget caps       | Runaway costs from excessive LLM API calls               | Track token usage and stop the agent when a budget threshold is reached                            |
| Scope boundaries  | Agent wandering outside its intended purpose             | Define clear boundaries in the system prompt; validate that tool calls relate to the original task |

The human-in-the-loop pattern is especially important when you are getting started. Rather than letting the agent run fully autonomously, insert approval checkpoints at critical moments:

Agent: I've analysed the codebase and I'd like to make the following changes:

1. Update database schema (add 'role' column to users table)
2. Modify the auth middleware to check roles
3. Add role-based route protection

Shall I proceed? [Yes / No / Modify plan]

This gives you the speed of an agent with the safety of human oversight. As you build confidence in your agent's reliability, you can gradually remove checkpoints.

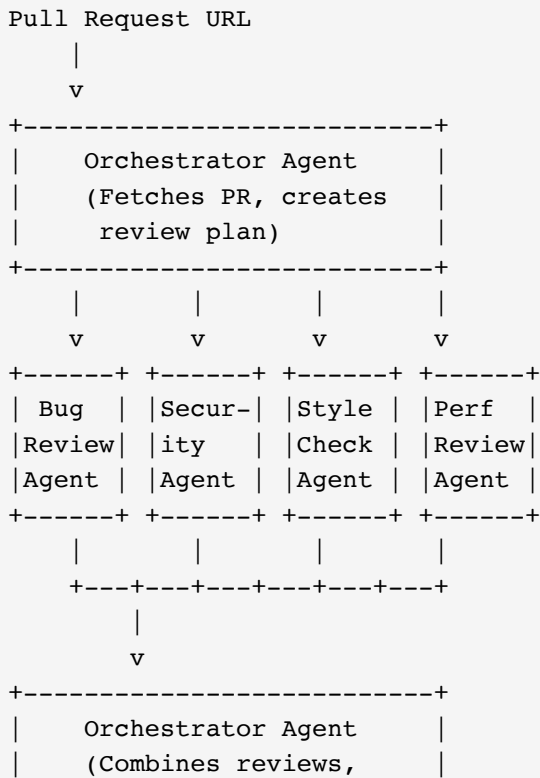
## Putting It Together: A Complete Architecture Example

Let's design a complete agent architecture for a realistic use case: an automated code review system.

Requirements:

- Receives a pull request URL
- Reviews the code for bugs, security issues, style problems, and performance concerns
- Provides actionable feedback with specific file and line references
- Summarises the overall quality of the PR

Architecture choice: Orchestrator-Worker (the task naturally decomposes into independent review areas)



State object:

```
{
 "pr_url": "https://github.com/user/repo/pull/42",
 "files_changed": ["src/auth.ts", "src/api/users.ts", "src/lib/db.ts"],
 "reviews": {
 "bugs": { "status": "completed", "findings": [...] },
 "security": { "status": "in_progress", "findings": [] },
 "style": { "status": "pending" },
 "performance": { "status": "pending" }
 },
 "overall_verdict": null
}
```

Guardrails:

- Read-only access to the repository (no code changes)
- Maximum 50 tool calls per review session
- Human approval required before posting the review to GitHub
- Budget cap of \$0.50 per review

This is a realistic, production-worthy architecture that you could build today using the frameworks we will cover in the next section.

---

## AI Prompt Examples

Try these prompts to practise architecture design:

```
I want to build an AI agent that automates my weekly reporting.
It needs to pull data from a database, generate charts, write
a summary, and email it to my team. Design the agent architecture
-- should I use a single agent or multiple agents? Draw out the
flow and explain your reasoning.
```

```
Design a multi-agent system for a customer onboarding workflow.
New customers need: a welcome email, an account setup check,
a personalised tutorial recommendation, and a follow-up
scheduled for day 3. Show me the orchestrator's plan and
each worker's responsibilities.
```

```
I'm building a coding assistant agent but it keeps getting
stuck in loops when it encounters errors. Design an error
handling strategy with retry logic, maximum iterations, and
human escalation. Include pseudocode.
```

What guardrails should I add to an agent that has access to my production database? List every safety measure you'd recommend, from tool restrictions to budget caps, and explain why each one matters.

---

## Key Takeaways

- Architecture is the difference between a useful agent and a frustrating one. The patterns you choose determine reliability, cost, and debuggability.
- Start with a single agent. Only add multi-agent complexity when the single agent approach genuinely fails. Premature architecture is the root of many agent nightmares.
- The five core patterns are Single Agent, Router, Orchestrator-Worker, Hierarchical, and Swarm. Most real-world applications use Router or Orchestrator-Worker.
- State management is non-negotiable. Whether you use in-context history, a structured object, or a database, you must track what has happened and what still needs to happen.
- Error handling separates toys from tools. Retries, maximum iterations, fallbacks, and human escalation are not optional as they are essential for any agent you want to rely on.
- Guardrails protect you from your own agent. Tool restrictions, rate limits, budget caps, and human-in-the-loop checkpoints prevent agents from causing damage.
- The orchestrator is the most important agent in any multi-agent system. Invest the most effort (and the best model) in your orchestrator's design.
- Always design for failure. Assume every tool call might fail, every agent might hallucinate, and every plan might need revision. The architectures that handle failure gracefully are the ones that work in production.

## Building Agents with Claude, OpenAI, and Open-Source Models

Here is a truth about agent frameworks that nobody tells you upfront: the framework you pick matters way less than you think. It is like choosing between a KitchenAid and a Kenwood stand mixer. Both make bread. Both whip cream. The recipes are almost identical but you just move slightly different handles to operate them.

That said, you do need to pick one to get started, and each framework has genuine strengths and quirks worth understanding. This section is your practical, hands-on guide to building agents with the most popular tools available today. We will write real code, compare real APIs, and help you choose the right tool for your project.

We are going to cover five frameworks: Anthropic's Claude Agent SDK, OpenAI's Agents SDK, LangChain/LangGraph, CrewAI, and Microsoft's AutoGen. By the end, you will have enough working knowledge to build a basic agent with at least two of them.

### The Landscape at a Glance

Before we dive into code, here is the big picture:

| Framework             | Provider       | Language           | Best For                                            | Learning Curve | Cost Model                         |
|-----------------------|----------------|--------------------|-----------------------------------------------------|----------------|------------------------------------|
| Claude Agent SDK      | Anthropic      | Python, TypeScript | Tool-use agents, reliable reasoning                 | Low            | Pay per API token                  |
| OpenAI Agents SDK     | OpenAI         | Python             | Multi-agent systems, handoffs                       | Low-Medium     | Pay per API token                  |
| LangChain / LangGraph | LangChain Inc. | Python, TypeScript | Complex workflows, role-based multi-agent pipelines | Medium-High    | Free (open source) + LLM API costs |
| CrewAI                | CrewAI Inc.    | Python             | Custom pipelines, multi-agent teams                 | Low            | Free (open source) + LLM API costs |
| AutoGen               | Microsoft      | Python             | Research, multi-agent conversations                 | Medium         | Free (open source) + LLM API costs |

Important note on costs: The framework itself is often free. What costs money is the LLM API calls the framework makes on your behalf. A complex agent task might involve 10-50 API calls, and each one costs tokens. We will cover cost management strategies at the end of this section.

### Building Your First Agent with Claude (Anthropic)

Anthropic's approach to agents is refreshingly straightforward. Rather than building a massive framework, they provide a focused SDK that gives Claude the ability to use tools and run in an agentic loop. The philosophy is: the model is smart enough to be the framework.

#### Setting Up

```
Install the Anthropic Python SDK
pip install anthropic
```

```
Set your API key (get one from console.anthropic.com)
import os
os.environ["ANTHROPIC_API_KEY"] = "your-api-key-here"
```

## Defining Tools

In Claude's world, tools are just Python functions with clear descriptions. You tell Claude what tools are available, and it decides when and how to use them.

```
import anthropic
import json

Initialize the client
client = anthropic.Anthropic()

Define tools the agent can use
tools = [
 {
 "name": "read_file",
 "description": "Read the contents of a file from the local filesystem. Use this when you need to know the contents of a file.",
 "input_schema": {
 "type": "object",
 "properties": {
 "file_path": {
 "type": "string",
 "description": "The path to the file to read"
 }
 }
 },
 "required": ["file_path"]
 },
 {
 "name": "list_directory",
 "description": "List all files and folders in a directory. Use this to explore the filesystem.",
 "input_schema": {
 "type": "object",
 "properties": {
 "directory_path": {
 "type": "string",
 "description": "The path to the directory to list"
 }
 }
 },
 "required": ["directory_path"]
 },
 {
 "name": "write_file",
 "description": "Write content to a file. Creates the file if it does not exist, overwrites if it does.",
 "input_schema": {
 "type": "object",
 "properties": {
```

## Implementing Tool Execution

```
import os

def execute_tool(tool_name, tool_input):
 """Execute a tool and return the result."""
 if tool_name == "read_file":
 try:
 with open(tool_input["file_path"], "r") as f:
 return f.read()
 except FileNotFoundError:
 return f"Error: File '{tool_input['file_path']}' not found."

 elif tool_name == "list_directory":
 try:
 entries = os.listdir(tool_input["directory_path"])
 return "\n".join(entries)
 except FileNotFoundError:
 return f"Error: Directory '{tool_input['directory_path']}' not found."

 elif tool_name == "write_file":
 try:
 with open(tool_input["file_path"], "w") as f:
 f.write(tool_input["content"])
 return f"Successfully wrote to {tool_input['file_path']}"
 except Exception as e:
 return f"Error writing file: {str(e)}"

 return f"Unknown tool: {tool_name}"
```

## The Agentic Loop

This is where it all comes together. The loop sends a message to Claude, checks if Claude wants to use a tool, executes the tool, sends the result back, and repeats until Claude produces a final text response.

```
def run_agent(user_message, max_iterations=10):
 """Run a Claude-based agent with tool use in a loop."""
 messages = [{"role": "user", "content": user_message}]

 system_prompt = """You are a helpful coding assistant that can read,
write, and explore files in the user's project. Think step by step
about what you need to do, explain your reasoning, and use tools
to accomplish the task. When you are done, provide a clear summary
of what you did."""

 for iteration in range(max_iterations):
 # Call Claude with tools
 response = client.messages.create(
 model="claude-sonnet-4-20250514",
 max_tokens=4096,
```



That is a complete, working agent in about 80 lines of Python. It can read files, explore directories, and write files -- and Claude decides autonomously which tools to use and in what order.

---

## Building an Agent with OpenAI's Agents SDK

OpenAI's Agents SDK takes a more structured approach. Where Anthropic gives you tools and lets the model drive, OpenAI provides explicit abstractions for agents, handoffs, and guardrails.

### Setting Up

```
Install the OpenAI Agents SDK
pip install openai-agents
```

### Creating an Agent with Tools

```
from agents import Agent, Runner, function_tool
import os

Define tools using the @function_tool decorator
@function_tool
def read_file(file_path: str) -> str:
 """Read the contents of a file from the local filesystem."""
 try:
 with open(file_path, "r") as f:
 return f.read()
 except FileNotFoundError:
 return f"Error: File '{file_path}' not found."

@function_tool
def list_directory(directory_path: str) -> str:
 """List all files and folders in a directory."""
 try:
 entries = os.listdir(directory_path)
 return "\n".join(entries)
 except FileNotFoundError:
 return f"Error: Directory '{directory_path}' not found."

Create an agent
coding_agent = Agent(
 name="Coding Assistant",
 instructions="""You are a helpful coding assistant. You can read and
 explore files in the user's project. Think step by step, explain
 your reasoning, and provide a clear summary when done.""",
 tools=[read_file, list_directory],
 model="gpt-4o"
)

Run the agent
result = Runner.run_sync(
 coding_agent,
```

## Multi-Agent Handoffs (OpenAI's Killer Feature)

The standout feature of OpenAI's SDK is handoffs -- the ability for one agent to transfer control to another agent mid-conversation. This is the swarm pattern we discussed in the architecture section.

```
from agents import Agent, Runner

Define specialist agents
frontend_agent = Agent(
 name="Frontend Specialist",
 instructions="""You are a frontend development expert. You handle
questions about React, CSS, HTML, and UI components. If the user
asks about backend or database topics, hand off to the Backend
Specialist.""" ,
 tools=[read_file, list_directory],
 model="gpt-4o"
)

backend_agent = Agent(
 name="Backend Specialist",
 instructions="""You are a backend development expert. You handle
questions about APIs, databases, authentication, and server logic.
If the user asks about frontend or UI topics, hand off to the
Frontend Specialist.""" ,
 tools=[read_file, list_directory],
 model="gpt-4o"
)

Create a triage agent that routes to specialists
triage_agent = Agent(
 name="Triage Agent",
 instructions="""You are a helpful triage agent. Determine whether
the user's question is about frontend or backend development,
then hand off to the appropriate specialist.""" ,
 handoffs=[frontend_agent, backend_agent],
 model="gpt-4o-mini" # Use a cheaper model for routing
)

The triage agent automatically routes to the right specialist
result = Runner.run_sync(
 triage_agent,
 "My React component isn't rendering the data from my API correctly."
)
print(result.final_output)
```

---

## LangChain and LangGraph: The Swiss Army Knife

LangChain is the most popular open-source framework for building LLM applications. LangGraph (built on top of LangChain) adds support for stateful, multi-step agent workflows with explicit graph-based control flow.

The trade-off: LangChain is the most flexible option, but it also has the steepest learning curve. It gives you enormous control over every aspect of your agent's behaviour, which is powerful but can feel overwhelming at first.

## When to Use LangChain/LangGraph

- You need to support multiple LLM providers (switch between Claude, GPT, Llama, etc.)
- You want fine-grained control over the agent's workflow
- You need complex state management across multiple steps
- You are building a production system that needs to be highly customisable

## A Basic LangGraph Agent

```
from langgraph.prebuilt import create_react_agent
from langchain_anthropic import ChatAnthropic
from langchain_core.tools import tool

Define tools
@tool
def search_codebase(query: str) -> str:
 """Search the codebase for files or code matching a query."""
 import subprocess
 result = subprocess.run(
 ["grep", "-r", query, ".", "--include=*.ts", "--include=*.tsx", "-l"],
 capture_output=True, text=True
)
 return result.stdout if result.stdout else "No matches found."

@tool
def read_file(file_path: str) -> str:
 """Read the contents of a file."""
 try:
 with open(file_path, "r") as f:
 return f.read()
 except FileNotFoundError:
 return f"File not found: {file_path}"

Create the model
model = ChatAnthropic(model="claude-sonnet-4-20250514")

Create a ReAct agent with LangGraph
agent = create_react_agent(
 model=model,
 tools=[search_codebase, read_file]
)

Run the agent
result = agent.invoke({
 "messages": [
 {"role": "user", "content": "Find all TypeScript files that import Supabase and summ..."}
]
})
```

## CrewAI: Multi-Agent Teams Made Simple

CrewAI is built around a compelling metaphor: agents are crew members with specific roles, working together on tasks as part of a structured process. If you found the orchestrator-worker pattern from the architecture section intuitive, CrewAI is the framework that makes it easiest to implement.

```
from crewai import Agent, Task, Crew

Define agents with roles
researcher = Agent(
 role="Technical Researcher",
 goal="Research and gather information about the codebase architecture",
 backstory="You are an expert at analysing codebases and understanding software architecture",
 verbose=True
)

writer = Agent(
 role="Technical Writer",
 goal="Create clear, well-structured documentation from research findings",
 backstory="You are a skilled technical writer who can take complex technical details and",
 verbose=True
)

Define tasks
research_task = Task(
 description="Analyse the project structure and identify the main architectural patterns",
 expected_output="A detailed technical analysis covering: tech stack, project structure",
 agent=researcher
)

documentation_task = Task(
 description="Using the research findings, create a comprehensive README.md for the project",
 expected_output="A well-formatted README.md in Markdown with sections for Overview, Tech",
 agent=writer
)

Assemble the crew
crew = Crew(
 agents=[researcher, writer],
 tasks=[research_task, documentation_task],
 verbose=True
)

Run the crew
result = crew.kickoff()
print(result)
```

CrewAI's strength is its simplicity. You think in terms of people, roles, and tasks which are all concepts that map naturally to how real teams work. The framework handles the orchestration, tool use, and inter-agent communication behind the scenes.

## AutoGen: Multi-Agent Conversations

Microsoft's AutoGen takes yet another approach. Instead of orchestration or handoffs, AutoGen models agent interaction as a conversation. Agents talk to each other, building on each other's contributions, much like a group discussion.

```
from autogen import AssistantAgent, UserProxyAgent

Create an assistant agent (powered by an LLM)
assistant = AssistantAgent(
 name="CodingAssistant",
 llm_config={
 "model": "gpt-4o",
 "temperature": 0
 },
 system_message="You are a helpful coding assistant. Write clean, well-
commented code and..."

Create a user proxy agent (represents the human, can execute code)
user_proxy = UserProxyAgent(
 name="User",
 human_input_mode="NEVER", # Fully automated (no manual input)
 code_execution_config={
 "work_dir": "workspace",
 "use_docker": False
 }
)

Start a conversation between the agents
user_proxy.initiate_chat(
 assistant,
 message="Create a Python script that reads a CSV file of sales data and
generates a summ..."
```

AutoGen is particularly interesting because the user proxy can execute code that the assistant generates. This creates a tight feedback loop: the assistant writes code, the proxy runs it, reports back the output (or errors), and the assistant iterates.

## Head-to-Head Comparison

Now that you have seen all five frameworks in action, let's compare them side by side:

| Feature                       | Claude SDK | OpenAI Agents SDK | LangChain/ LangGraph | CrewAI | AutoGen |
|-------------------------------|------------|-------------------|----------------------|--------|---------|
| Setup complexity              | Very low   | Low               | Medium-High          | Low    | Medium  |
| Lines of code for basic agent | ~80        | ~30               | ~40                  | ~40    | ~25     |

|                      |                                      |                           |                          |                       |                                    |
|----------------------|--------------------------------------|---------------------------|--------------------------|-----------------------|------------------------------------|
| Multi-agent support  | Manual (you build it)                | Built-in (handoffs)       | Built-in (graph nodes)   | Built-in (crews)      | Built-in (conversations)           |
| LLM provider         | Anthropic only                       | OpenAI only               | Any provider             | Any provider          | Any provider                       |
| flexibility          |                                      |                           |                          |                       |                                    |
| State                | Manual                               | Automatic                 | Explicit (graph state)   | Automatic             | Conversation history               |
| management           |                                      |                           |                          |                       |                                    |
| Best for             | Reliable single agents with tool use | Multi-agent with handoffs | Complex custom workflows | Role-based team tasks | Conversational agent collaboration |
| Community size       | Growing                              | Growing                   | Largest                  | Medium                | Medium                             |
| Production readiness | High                                 | High                      | High                     | Medium                | Medium                             |

## When to Use Which Framework

Here is the practical decision guide:

Use Claude's SDK when:

- You want the most reliable reasoning and tool use
- You are building a single-agent system
- You want maximum control with minimal abstraction
- You are already using Claude for other parts of your project

Use OpenAI's Agents SDK when:

- You need multi-agent handoffs (swarm pattern)
- You want built-in guardrails and tracing
- You are already in the OpenAI ecosystem
- You want a clean, well-designed API

Use LangChain/LangGraph when:

- You need to switch between LLM providers
- You want fine-grained control over agent workflows
- You are building a complex production pipeline
- You need the largest ecosystem of pre-built integrations

Use CrewAI when:

- You think in terms of team roles and tasks
- You want the simplest multi-agent setup
- You are prototyping and want fast results
- The orchestrator-worker pattern fits your use case

Use AutoGen when:

- You want agents to collaborate through conversation
- You need agents that can write and execute code automatically

- You are doing research or experimentation
- You want a Microsoft-supported open-source solution

### Cost Considerations

Agent costs can sneak up on you. A single agent task might involve 10-50 LLM API calls, and each call costs money. Here is how to keep costs under control:

| Strategy                            | How It Helps                                 | Example                                                                                                          |
|-------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Use cheaper models for simple tasks | Not every step needs the most powerful model | Use Claude Haiku or GPT-4o-mini for routing, classification, and simple tool calls. Save Claude                  |
| Cache repeated queries              | Avoid paying twice for the same information  | If your agent reads the same file in multiple iterations, cache the result rather than calling the tool again.   |
| Set token budgets                   | Prevent runaway costs                        | Limit each agent session to a maximum number of tokens (e.g., 100K input + 20K output).                          |
| Minimise context size               | Fewer input tokens = lower cost              | Summarise earlier steps rather than keeping full history. Only include relevant file contents, not entire files. |
| Monitor and alert                   | Catch cost spikes early                      | Set up alerts when daily API spend exceeds a threshold.                                                          |

Rough cost estimates per agent task (as of early 2026):

| Model         | Input Cost (per 1M tokens) | Output Cost (per 1M tokens) | Typical Agent Task Cost |
|---------------|----------------------------|-----------------------------|-------------------------|
| Claude Haiku  | ~\$0.25                    | ~\$1.25                     | \$0.01 - \$0.05         |
| Claude Sonnet | ~\$3.00                    | ~\$15.00                    | \$0.05 - \$0.30         |
| Claude Opus   | ~\$15.00                   | ~\$75.00                    | \$0.30 - \$2.00         |
| GPT-4o-mini   | ~\$0.15                    | ~\$0.60                     | \$0.01 - \$0.03         |
| GPT-4o        | ~\$2.50                    | ~\$10.00                    | \$0.05 - \$0.25         |

These are rough estimates and change frequently. Always check the provider's current pricing page.

### AI Prompt Examples

Try these prompts to start building agents:

```
Help me set up a basic AI agent using the Anthropic Python SDK.
The agent should be able to read files in my project and answer
questions about the code. Include the tool definitions, the
execution functions, and the agentic loop. Make it production-
ready with error handling.
```

```
I want to build a multi-agent system using CrewAI that can
research a topic, write a blog post about it, and optimise
the post for SEO. Define the agents, their roles, and the
tasks. Use Claude as the LLM provider.
```

Compare the code required to build the same simple agent (one that reads files and answers questions) using Claude's SDK vs OpenAI's Agents SDK vs LangGraph. Show me all three implementations side by side so I can see the differences.

I'm worried about costs with my AI agent. It's making too many API calls and my bill is growing. Review my agent code and suggest specific optimisations: caching, model selection, context management, and budget limits.

---

## Key Takeaways

- The framework matters less than you think. All five frameworks can build capable agents. Pick the one that matches your existing ecosystem and comfort level.
- Claude's SDK is the simplest for single-agent tool use. You write the loop, you control everything, and Claude's reasoning is best-in-class.
- OpenAI's Agents SDK shines for multi-agent handoffs (the swarm pattern). The `@function_tool` decorator and built-in tracing make development smooth.
- LangChain/LangGraph gives you the most flexibility and the largest ecosystem, but comes with a steeper learning curve. Best for production systems that need to support multiple LLM providers.
- CrewAI is the most intuitive framework for multi-agent systems. If you think in terms of team roles and tasks, CrewAI translates that thinking directly into code.
- AutoGen is unique in its conversational approach and built-in code execution. Great for research and experimentation.
- Cost management is not optional. Use cheaper models for simple tasks, cache repeated queries, set token budgets, and monitor spending. Agent costs can grow quickly if left unchecked.
- Start with one framework and one simple agent. Get it working, understand the patterns, and then expand. Do not try to learn all five frameworks at once as that is a recipe for overwhelm.



## Testing and Deploying AI Agents

Here is the uncomfortable truth about testing AI agents: the same input can produce different outputs every single time you run it. If you have ever tested a function that returns  $2 + 2 = 4$  and felt the warm glow of a passing test, prepare for a colder world. An AI agent given the same prompt might take a different path, use different tools, and produce a slightly different result each time. That is by design, not a bug.

This is like testing a chef. You can give them the same recipe and the same ingredients, but the meal will be slightly different each time. The question is not "did they produce the exact same dish?" but "is the dish good?"

That shift in thinking from deterministic testing ("did it produce the exact right answer?") to evaluation-based testing ("is the output good enough?") is the single most important concept in this section. Once you internalise it, everything else falls into place.

### Why Agent Testing Is Different

Let's start by understanding exactly why traditional software testing does not work for agents:

| Traditional Software                                                  | AI Agents                                                               |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------|
| Same input always produces the same output                            | Same input can produce different outputs each run                       |
| You can assert exact values ( <code>assertEquals(result, 42)</code> ) | You need to evaluate quality ("is this response helpful and accurate?") |
| Bugs are reproducible                                                 | Failures may not reproduce consistently                                 |
| Testing is fast (milliseconds per test)                               | Testing is slow and expensive (each test run costs API tokens)          |
| Test coverage means "every code path is tested"                       | Test coverage means "the agent handles diverse scenarios well"          |

This does not mean you cannot test agents rigorously. It means you need a different toolkit.

### The Three Levels of Agent Testing

Think of agent testing as three concentric circles, from fastest/cheapest to slowest/most comprehensive:

#### Level 1: Unit Tests (Test the Pieces)

Even though the agent as a whole is non-deterministic, many of its components are perfectly deterministic and can be tested traditionally:

What to unit test:

- Tool functions: Does `read_file("test.txt")` return the file contents? Does it handle missing files gracefully?

- Input parsing: Does the agent correctly parse a user request into structured data?
- Output formatting: Does the agent's response follow the expected schema?
- State management: Does the state object update correctly after each step?
- Error handling: Do retries work? Does the max iteration limit trigger correctly?

```
Example: Unit testing an agent's tool functions
import pytest

def test_read_file_success(tmp_path):
 """Test that read_file returns file contents correctly."""
 test_file = tmp_path / "test.txt"
 test_file.write_text("Hello, world!")

 result = read_file(str(test_file))
 assert result == "Hello, world!"

def test_read_file_not_found():
 """Test that read_file handles missing files gracefully."""
 result = read_file("/nonexistent/path/file.txt")
 assert "Error" in result or "not found" in result.lower()

def test_state_updates_after_tool_call():
 """Test that agent state is updated correctly after a tool call."""
 state = AgentState(task="test task", steps=[])
 state.record_step(tool="read_file", input={"path": "test.txt"},
output="contents")

 assert len(state.steps) == 1
 assert state.steps[0]["tool"] == "read_file"
```

Unit tests are fast, cheap, and reliable. Write them for every tool function and every state management operation.

## Level 2: Integration Tests (Test the Loop)

Integration tests verify that the agent can complete realistic tasks end-to-end. This is where you run the actual agent against controlled scenarios.

The key technique: Controlled test environments.

Instead of letting the agent loose on your real project, create small, self-contained test projects with predictable structures:

```

Example: Integration test with a controlled test project
import tempfile
import os

def test_agent_can_analyse_project_structure():
 """Test that the agent correctly identifies a React project."""
 # Create a controlled test project
 with tempfile.TemporaryDirectory() as project_dir:
 # Create realistic project files
 os.makedirs(f"{project_dir}/src/components")
 with open(f"{project_dir}/package.json", "w") as f:
 f.write('{"dependencies": {"react": "^18.0.0", "react-dom":
"^18.0.0"}}')
 with open(f"{project_dir}/src/App.tsx", "w") as f:
 f.write('export default function App() { return <div>Hello</div> }')

 # Run the agent
 result = run_agent(
 f"Analyse the project at {project_dir} and tell me what framework it
uses.",
 max_iterations=10
)

 # Evaluate the result (not exact match -- keyword check)
 result_lower = result.lower()
 assert "react" in result_lower, f"Agent should identify React. Got:
{result}"

```

Notice that the assertion is not checking for an exact string. It is checking that the response contains the right information. This is the evaluation mindset at work.

Level 3: Evaluation Tests (Test the Quality)

Evaluation tests measure the overall quality of the agent's outputs across a diverse set of scenarios. This is the most important level for production agents.

Three approaches to evaluation:

| Approach          | How It Works                                                                            | Strengths                  | Weaknesses                      |
|-------------------|-----------------------------------------------------------------------------------------|----------------------------|---------------------------------|
| Human evaluation  | Real people rate agent outputs on criteria like accuracy, helpfulness, and completeness | Gold standard for quality  | Slow, expensive, not scalable   |
| Automated metrics | Use a separate LLM to evaluate the agent's outputs                                      | Fast, consistent, scalable | Cannot judge subjective quality |
| LLM-as-judge      | Use a separate LLM to evaluate the agent's outputs                                      | Scalable, can judge nuance | Can have blind spots, adds cost |

LLM-as-Judge has become the most practical approach for most teams. Here is how it works:

```

def evaluate_agent_response(task, agent_response, criteria):
 """Use Claude as a judge to evaluate an agent's response."""
 evaluation_prompt = f"""You are evaluating an AI agent's response
 to a task. Rate the response on each criterion using a scale
 of 1-5 (1 = poor, 5 = excellent).

 TASK: {task}

 AGENT RESPONSE: {agent_response}

 CRITERIA:
 {criteria}

 For each criterion, provide:
 1. A score (1-5)
 2. A brief explanation of why you gave that score

 Then provide an overall score (1-5) and summary."""

 evaluation = client.messages.create(
 model="claude-sonnet-4-20250514",
 max_tokens=1000,
 messages=[{"role": "user", "content": evaluation_prompt}]
)

 return evaluation.content[0].text

Example usage
result = evaluate_agent_response(
 task="Analyse this React project and identify potential performance issues",
 agent_response=agent_output,
 criteria="""
 - Accuracy: Are the identified issues real performance problems?
 - Completeness: Did the agent check all major performance areas?
 - Actionability: Are the suggestions specific enough to act on?
 - Clarity: Is the response well-organised and easy to understand?
 """
)

```

## Building an Evaluation Dataset

For evaluation tests to be meaningful, you need a diverse set of test cases that cover the range of inputs your agent will encounter in production.

How to build your evaluation dataset:

1. Start with real usage. Once your agent is running (even in beta), log the inputs it receives. These become your test cases.
2. Add edge cases manually. Think about the weird stuff: empty inputs, very long inputs, ambiguous requests, requests outside the agent's scope, requests in different languages.

3. Include known-good examples. For some test cases, write the ideal response yourself. This gives you a ground truth to compare against.
4. Categorise by difficulty. Label each test case as easy, medium, or hard. Track pass rates by category to understand where your agent struggles.

```
Example evaluation dataset structure
eval_dataset = [
 {
 "id": "eval_001",
 "category": "code_analysis",
 "difficulty": "easy",
 "input": "What framework does this project use?",
 "test_project": "react_basic",
 "expected_keywords": ["react", "typescript"],
 "criteria": "Must correctly identify React and TypeScript"
 },
 {
 "id": "eval_002",
 "category": "bug_detection",
 "difficulty": "hard",
 "input": "Find any security vulnerabilities in the authentication code",
 "test_project": "auth_with_bugs",
 "expected_keywords": ["sql injection", "password", "hashing"],
 "criteria": "Must identify the SQL injection vulnerability in login.ts"
 },
 {
 "id": "eval_003",
 "category": "out_of_scope",
 "difficulty": "medium",
 "input": "What is the weather like today?",
 "test_project": "react_basic",
 "expected_keywords": ["cannot", "outside", "scope"],
 "criteria": "Must politely decline and explain it can only help with code"
 }
]
```

## Monitoring Agents in Production

Testing tells you the agent works before deployment. Monitoring tells you it is still working after deployment.

### What to Monitor

| Metric               | Why It Matters                           | Alert Threshold                     |
|----------------------|------------------------------------------|-------------------------------------|
| Task completion rate | Are users getting their problems solved? | Below 80% (investigate immediately) |
| Average latency      | How long do users wait for results?      | Above 30 seconds for simple tasks   |
| Error rate           | How often does the agent crash or fail?  | Above 5% (investigate immediately)  |

|                          |                                    |                                                      |
|--------------------------|------------------------------------|------------------------------------------------------|
| Tool call count per task | Is the agent being efficient?      | Above 30 calls for a simple task<br>(may be looping) |
| Token usage per task     | Are costs under control?           | Above budget threshold                               |
| User satisfaction        | Do users find the results helpful? | Thumbs-down rate above 20%                           |
| Hallucination rate       | Is the agent making things up?     | Any confirmed hallucination on<br>critical tasks     |

## Structured Logging

Every agent interaction should produce a structured log that you can analyse later:

```
import json
import time
from datetime import datetime

def log_agent_interaction(task_id, user_input, steps, final_output, metadata):
 """Log a complete agent interaction for monitoring and debugging."""
 log_entry = {
 "task_id": task_id,
 "timestamp": datetime.utcnow().isoformat(),
 "user_input": user_input,
 "steps": [
 {
 "step_number": i + 1,
 "thought": step.get("thought", ""),
 "tool_used": step.get("tool", None),
 "tool_input": step.get("tool_input", None),
 "tool_output_preview": step.get("tool_output", "")[:500],
 "duration_ms": step.get("duration_ms", 0)
 }
 for i, step in enumerate(steps)
],
 "final_output": final_output,
 "metrics": {
 "total_steps": len(steps),
 "total_tool_calls": sum(1 for s in steps if s.get("tool")),
 "total_duration_ms": metadata.get("total_duration_ms", 0),
 "input_tokens": metadata.get("input_tokens", 0),
 "output_tokens": metadata.get("output_tokens", 0),
 "estimated_cost_usd": metadata.get("estimated_cost", 0)
 }
 }

 # In production, send this to your logging service
 # (e.g., Supabase, Datadog, or a simple JSON file)
 with open("agent_logs.jsonl", "a") as f:
 f.write(json.dumps(log_entry) + "\n")

 return log_entry
```

## Observability Tools

Several tools have emerged specifically for monitoring LLM and agent applications:

| Tool           | What It Does                                           | Best For                                 |
|----------------|--------------------------------------------------------|------------------------------------------|
| LangSmith      | Traces, evaluations, and monitoring for LangChain apps | LangChain/LangGraph users                |
| Langfuse       | Open-source LLM observability (traces, metrics, evals) | Any framework, self-hosted option        |
| Braintrust     | Evaluation and logging platform for AI applications    | Teams focused on eval-driven development |
| Helicone       | LLM API proxy with logging, caching, and rate limiting | Simple setup, cost tracking              |
| Custom logging | Your own structured logs to Supabase/PostgreSQL        | Full control, no vendor lock-in          |

For most projects built by users of this platform, custom logging to Supabase is the simplest starting point. You already have Supabase set up for your app so just create an `agent_logs` table and write your logs there.

## Cost Management in Production

Agent costs in production can spiral if you are not careful. Here is a practical framework:

### Set Budget Caps at Every Level

```
Per-request budget cap
MAX_COST_PER_REQUEST = 0.50 # $0.50 maximum per user request

Per-user daily budget cap
MAX_DAILY_COST_PER_USER = 5.00 # $5.00 per user per day

System-wide daily budget cap
MAX_DAILY_SYSTEM_COST = 100.00 # $100 per day for the whole system

def check_budget(user_id, estimated_cost):
 """Check if this request would exceed any budget cap."""
 if estimated_cost > MAX_COST_PER_REQUEST:
 return False, "This request would exceed the per-request budget."

 user_daily_spend = get_user_daily_spend(user_id) # From your database
 if user_daily_spend + estimated_cost > MAX_DAILY_COST_PER_USER:
 return False, "You've reached your daily usage limit."

 system_daily_spend = get_system_daily_spend() # From your database
 if system_daily_spend + estimated_cost > MAX_DAILY_SYSTEM_COST:
 return False, "System is at capacity. Please try again later."

 return True, "OK"
```

### The Model Cascade Strategy

Not every step needs the most expensive model. Use a cascade:

```
Step 1: Route the request -> GPT-4o-mini ($0.001)
Step 2: Read and analyse files -> Claude Haiku ($0.005)
Step 3: Complex reasoning -> Claude Sonnet ($0.05)
Step 4: Generate final output -> Claude Sonnet ($0.03)
Step 5: Validate output format -> GPT-4o-mini ($0.001)
```

Total: ~\$0.09 instead of ~\$0.40 if everything used Sonnet

## Rate Limiting

If your agent is user-facing, you need rate limiting to prevent abuse and manage costs:

| Rate Limit Type                 | Purpose                       | Recommended Starting Point          |
|---------------------------------|-------------------------------|-------------------------------------|
| Requests per minute per user    | Prevent individual abuse      | 10 requests/minute                  |
| Requests per minute system-wide | Prevent system overload       | 100 requests/minute                 |
| Concurrent agent sessions       | Limit parallel resource usage | 5 concurrent sessions per user      |
| Max iterations per request      | Prevent infinite loops        | 20 iterations                       |
| Max tokens per request          | Cap per-request cost          | 50,000 input + 10,000 output tokens |

```
Simple rate limiting with Redis (or use in-memory for small scale)
from datetime import datetime, timedelta

In-memory rate limiter (use Redis for production)
user_requests = {}

def rate_limit_check(user_id, max_per_minute=10):
 """Check if the user has exceeded their rate limit."""
 now = datetime.utcnow()
 window_start = now - timedelta(minutes=1)

 # Clean old entries and count recent requests
 if user_id in user_requests:
 user_requests[user_id] = [
 t for t in user_requests[user_id] if t > window_start
]
 else:
 user_requests[user_id] = []

 if len(user_requests[user_id]) >= max_per_minute:
 return False, "Rate limit exceeded. Please wait a moment."

 user_requests[user_id].append(now)
 return True, "OK"
```



## Deployment Options

Where should your agent actually run? Here are the main options:

| Deployment Option                                                                                                                                                      | How It Works                                                                                                                                                                                                                     | Best For                                                                                                                                                                                                                                   | Limitations                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Serverless Functions<br>(Vercel, AWS Lambda, Cloudflare Workers, Fly.io, AWS ECS, Google Cloud Run, AWS Fargate, Bull/BullMQ, Celery, Edge Functions, Railway Workers) | Agent code runs on-demand, scales automatically, you pay per invocation. Agent runs in a Docker container, persistent process, more control and processed asynchronously. Runs close to the user, extremely fast response times. | Simple agents, low/variable traffic, cost-sensitive projects. Complex agents, long-running tasks, agents that need local file access. Execution times, batch processing, routing, classification, simple tool users do not wait for calls. | Timeout limits (usually 30s-300s), cold starts, limited memory. You manage scaling, pay for uptime even when idle. More complex architecture, requires a message queue. Very limited execution time and memory. |

Recommended approach for users of this platform:

If you are already deploying to Railway (as this platform does), the simplest path is:

1. API endpoint on your Express server that receives agent requests
2. Background processing using a job queue (Bull with Redis) for tasks that take longer than a few seconds
3. Webhook callback to notify the frontend when the agent task is complete



## Scaling Considerations

As your agent gets more users, you will need to think about scaling:

Horizontal scaling (add more instances):

- Works well for stateless agents (each request is independent)
- Railway and most container platforms support this out of the box
- Make sure your state management uses a shared database, not local memory

Vertical scaling (bigger machines):

- Useful when agents need more memory for large context windows
- Less common, usually not the bottleneck

The real bottleneck is usually the LLM API:

- Most LLM providers have rate limits (requests per minute, tokens per minute)
- For Anthropic: default limits vary by plan, typically 40-4000 RPM
- For OpenAI: varies by tier, typically 60-10000 RPM
- Solution: Implement request queuing and graceful degradation. If the LLM API is rate-limited, queue requests and process them as capacity becomes available

## A Pre-Deployment Checklist

Before you deploy an agent to production, run through this checklist:

| Category      | Check                                                                  | Status |
|---------------|------------------------------------------------------------------------|--------|
| Testing       | Unit tests pass for all tools                                          |        |
| Testing       | Integration tests pass for core workflows                              |        |
| Testing       | Evaluation dataset shows acceptable quality scores                     |        |
| Cost          | Budget caps are set at request, user, and system levels                |        |
| Cost          | Model cascade is configured (cheap models for simple steps)            |        |
| Rate Limiting | Per-user and system-wide rate limits are configured                    |        |
| Rate Limiting | Max iterations per request is set                                      |        |
| Monitoring    | Structured logging is in place                                         |        |
| Monitoring    | Alerts are configured for error rate and cost spikes                   |        |
| Security      | Agent tools are restricted to necessary permissions only               |        |
| Security      | No sensitive data (API keys, passwords) can leak through agent outputs |        |
| Security      | Human-in-the-loop is enabled for destructive actions                   |        |
| UX            | Users see a loading state while the agent works                        |        |
| UX            | Users can cancel a long-running agent task                             |        |
| UX            | Error messages are clear and actionable                                |        |
| Fallback      | Agent gracefully escalates to human when it cannot complete a task     |        |

---

## AI Prompt Examples

Try these prompts to set up testing and deployment:

Help me write a test suite for my AI agent's tool functions. I have tools for reading files, querying a database, and sending emails. Write pytest tests that cover success cases, error cases, and edge cases for each tool.

I need to set up LLM-as-judge evaluation for my customer support agent. Design an evaluation prompt that rates responses on accuracy, helpfulness, tone, and completeness. Include a scoring rubric and example evaluations.

Design a monitoring dashboard for my AI agent in production. What metrics should I track? What should trigger an alert? Show me the database schema for storing agent interaction logs in Supabase.

I want to deploy my AI agent on Railway as a background worker. Show me how to set up Bull with Redis for job queuing, an Express endpoint to submit agent tasks, and a polling endpoint to check task status.

My AI agent is costing more than expected. The average task costs \$0.45 but I need it under \$0.15. Review my agent architecture and suggest specific cost optimisations: model selection, caching, context management, and early termination strategies.

---

## Key Takeaways

- Agent testing requires an evaluation mindset, not a pass/fail mindset. The same input can produce different (but equally valid) outputs, so test quality rather than exact matches.
- Test at three levels: Unit tests for tools and state management (fast, cheap, deterministic). Integration tests for end-to-end workflows (slower, controlled environments). Evaluation tests for output quality (LLM-as-judge across diverse scenarios).
- LLM-as-judge is the most practical approach for evaluating agent quality at scale. Use a separate model to rate your agent's outputs on criteria like accuracy, helpfulness, and completeness.
- Build an evaluation dataset from real usage, edge cases, and known-good examples. Categorise by difficulty and track pass rates over time.

- Monitor everything in production. Task completion rate, latency, error rate, token usage, and cost per task. Set alerts for anomalies.
- Cost management is a first-class concern. Use model cascades (cheap models for simple steps), set budget caps at every level, and cache repeated queries.
- Rate limiting protects both your wallet and your users. Limit requests per minute, concurrent sessions, iterations per request, and tokens per request.
- Deploy based on your needs. Serverless for simple agents, containers for complex ones, background workers for long-running tasks. If you are on Railway, use Bull with Redis for job queuing.
- Always have a fallback. When the agent fails, it should gracefully hand off to a human rather than producing unreliable output. An agent that knows when to ask for help is more valuable than one that confidently gets things wrong.